Algorithmic Foundations of Stacks: Abstract Data Types, Delimiter Matching Invariants, Postfix Arithmetic Evaluation, and Amortized Linear Monotonic Topographies

Abstract

This research paper presents an exhaustive pedagogical and systems-level formalization of the Stack abstract data type and its advanced algorithmic manifestations. Grounded in the Last-In, First-Out (LIFO) behavioral discipline, we analyze contiguous buffer mechanics, addressing stack underflow, overflow, and geometric capacity doubling. We establish the syntactic mechanics of nested delimiter validation, proving that the Last Opened Bracket (LOB) invariant governs the linear-time O(N) parsing of balanced multi-character parentheses sequences. We formalize deterministic parenthesis-free arithmetic computation via Postfix (Reverse Polish Notation) state machines, highlighting the critical non-commutative operand popping asymmetry. Furthermore, we demystify the Monotonic Stack paradigm through the Previous Smaller Element problem. By formulating the Elimination Lemma and the Skyline Shadowing Model, we prove why greater elements are permanently redundant, maintaining an invariant ascending valley stack. Through the Aggregate Potential Method, we resolve the apparent quadratic paradox of nested loops, proving an amortized O(N) runtime bound with O(1) per-element operations. Finally, we map the four-quadrant monotonic taxonomy across classical algorithmic benchmarks and examine real-world industrial architectures, including the JVM operand stack, browser dual-stack history traversal, and compiler AST syntax analysis. Compilable production implementations in Java, C++, and Python accompany all discussions.

Index Terms

Stack ADT, LIFO invariant, dynamic array resizing, balanced parentheses, delimiter matching, postfix evaluation, reverse Polish notation, monotonic stack, nearest smaller element, amortized O(N) analysis, compiler syntax parsing

1 Introduction: The Stack Abstract Data Type

The Stack is one of the most fundamental linear data structures in computer science, serving as the architectural backbone for runtime execution environments, compiler syntax parsers, memory management systems, and recursive algorithm evaluation. At its core, a stack enforces a strict behavioral discipline: Last-In, First-Out (LIFO).

In a LIFO structure, access is restricted to a single distinguished position termed the Top. The element most recently inserted into the collection is invariably the first entity eligible for inspection or removal.

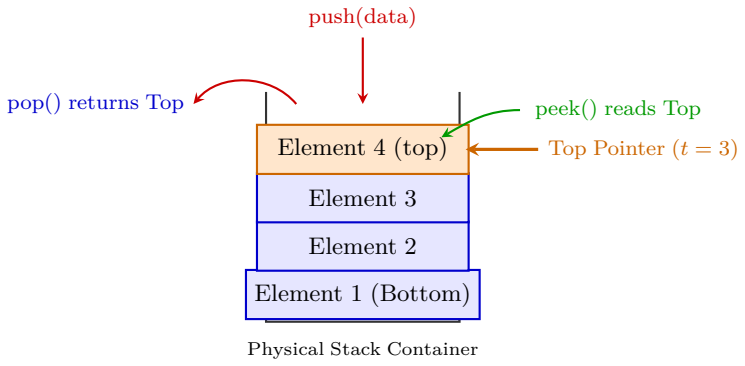


Fig. 1. The LIFO Architectural Model: Element insertion, inspection, and removal restricted to the top boundary.

1.1 Physical Mental Models and Computational Analogies

To grasp why the stack abstraction is pervasive across engineering, consider two standard mental models:

1) The Spring-Loaded Cafeteria Tray Dispenser: Clean trays are pushed onto the top of the stack, compressing the spring. Customers take trays exclusively from the top. The first tray placed at the bottom of the dispenser remains dormant until every single tray placed above it has been consumed.
2) Call Frame Allocation on the Execution Stack: When function main() invokes function foo(), which in turn invokes bar(), the runtime allocates stack frames in sequential order: [main → foo → bar]. Because bar() must complete its computation before foo() can resume, bar()'s frame must be deallocated (popped) first.

1.2 Formal Abstract Data Type (ADT) Specification

The formal Stack ADT defines a mathematical model along with a collection of primitive operations satisfying strict temporal guarantees:

TABLE 1
The Stack Abstract Data Type Core Operations

Operation	Input / Output	Complexity	Operational Semantics
push(x)	$x \in \mathcal{U} \to \emptyset$	$O(1)$	Inserts element x at the current top position; increments top index $t \leftarrow t + 1$.
pop()	$\emptyset \to x \in \mathcal{U}$	$O(1)$	Removes and returns the top element; decrements top index $t \leftarrow t - 1$. Underflows if empty.
peek()	$\emptyset \to x \in \mathcal{U}$	$O(1)$	Returns top element without modifying stack state. Underflows if empty.
isEmpty()	$\emptyset \to \{\text{true}, \text{false}\}$	$O(1)$	Evaluates whether stack contains zero elements ($t == -1$).
size()	$\emptyset \to \mathbb{Z}_{\geq 0}$	$O(1)$	Returns current cardinal count of stored elements ($t + 1$).

1.3 Contiguous Array Implementation Mechanics

A stack can be implemented over a contiguous pre-allocated array $A[0 \dots C-1]$ of fixed capacity C. A single integer variable t (the top index pointer) tracks the current boundary:

- Initialization: The stack is initialized empty with $t = -1$.
- Stack Overflow Boundary: If a push operation is triggered when $t = C - 1$, the buffer has reached maximum capacity. In fixed systems, this raises a Stack Overflow Exception. In dynamic memory vectors (e.g., ArrayList or std::vector), the underlying buffer is dynamically doubled to $2C$, copying existing elements in amortized $O(1)$ time.
- Stack Underflow Boundary: If pop() or peek() is triggered when $t = -1$ (i.e., isEmpty() is true), attempting to index $A[-1]$ represents illegal memory dereferencing. This condition is termed a Stack Underflow Exception.

2 Syntactic Invariant Validation: The Balanced Delimiter Problem

2.1 The Golden Invariant: Last Opened Bracket Closed First

In syntax trees, mathematical expressions, and programming compilers (e.g., parsing JSON, XML, Java, or C++ blocks), nested delimiters must open and close in strictly balanced configurations. Delimiters belong to matching pairs:

$$\mathcal{P} = \{(\text{`('}, \text{`)'}), (\text{`\{'}, \text{`\}'}), (\text{`['}, \text{`]'}')\}$$

The foundational invariant governing syntactic validity can be stated without complex automata notation:

The Golden Invariant (LOB): The Last Opened Bracket (LOB) must always be the First Bracket Closed.

Because the most recently encountered unclosed opening delimiter must be resolved before any older opening delimiter can be legally closed, delimiter validation maps directly onto the LIFO discipline.

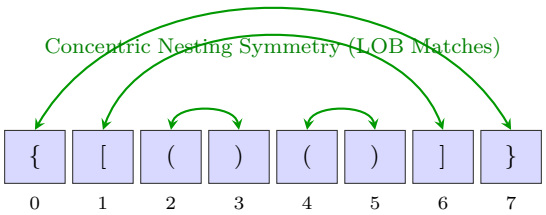


Fig. 2. Nested Delimiter Symmetry: Inner pairs resolve before outer pairs can close.

2.2 State Machine Parser Algorithm

Given a string S of length N:

1) Initialize an empty stack of characters $\mathcal{S}$.
2) Iterate through each character c in S from index 0 to $N-1$:
   - Case 1 (Opening Delimiter): If $c \in \{`(', `\{', `[']\}$, push c onto $\mathcal{S}$.
   - Case 2 (Closing Delimiter): If $c \in \{`)', `\}', `]'\}$:
     a) Underflow Failure: If $\mathcal{S}$.isEmpty() is true, the sequence contains an orphaned closing delimiter without a preceding open bracket. Return False.
     b) Mismatch Failure: Peek the top element $top = \mathcal{S}$.peek(). If top is not the corresponding opening delimiter for c (e.g., $top = `('$ but $c = `]'$), return False.
     c) Successful Match: Pop the matching opening delimiter: $\mathcal{S}$.pop().
3) Termination Invariant: Upon string exhaustion, the sequence is valid if and only if $\mathcal{S}$.isEmpty() is true. If $\mathcal{S}$ is non-empty, unclosed opening brackets remain. Return $\mathcal{S}$.isEmpty().

2.3 Exhaustive Dry-Run Verification Matrix

TABLE 2
Trace of Delimiter Matching Parser on Sample Configurations

Test Case	Input String	Steps	Stack State Transitions & Decision Logic	Valid?
1	"{[()]}"	6	$\emptyset \to [\{] \to [\{, [] \to [\{, [, (] \to$ pop() $\to$ pop[] $\to$ pop{} $\to \emptyset$ (Clean match)	True
2	"([)]"	3	$\emptyset \to [(] \to [(, [] \to$ encounter ')' $\neq$ peek '['. Type mismatch detected.	False
3	"(()"	3	$\emptyset \to [(] \to [(, (] \to$ pop(). Input exhausted with stack non-empty: $\mathcal{S} = [(]$.	False
4	"]()"	1	$\emptyset \to$ encounter ']' while stack is empty. Premature underflow detected.	False

Complexity Analysis: Each character is pushed at most once and popped at most once. Both time complexity and auxiliary space complexity are strictly $O(N)$.

3 Deterministic Arithmetic Computation: Postfix Expression Evaluation

3.1 Infix vs. Postfix (Reverse Polish Notation)

Standard arithmetic notation (e.g., $A + B \times C$) is Infix, meaning operators reside between their operands. Infix notation requires operator precedence hierarchies ($* > +$) and explicit parentheses to disambiguate evaluation order (e.g., $(A+B)\times C$).

In Postfix Notation (also known as Reverse Polish Notation, RPN), operators strictly follow their operands:

$$A + B \implies A\ B\ +$$
$$(A + B) \times C \implies A\ B\ +\ C\ \times$$

Architectural Advantage: In postfix form, expression evaluation is completely linear and parenthesis-free. The sequence of tokens encodes the exact temporal order in which operations must be executed, making RPN the standard bytecode format for virtual machines (such as the JVM operand stack and Forth environments).

3.2 Stack Evaluation State Machine

Given a tokenized postfix expression:

1) Initialize an operand stack $\mathcal{S}$.
2) Scan tokens sequentially from left to right:
   - Token is an Operand (Number): Push numerical value onto $\mathcal{S}$.
   - Token is an Operator ($\odot \in \{+, -, \times, /\}$):
     a) Pop operand $B = \mathcal{S}$.pop() (Right/Second Operand).
     b) Pop operand $A = \mathcal{S}$.pop() (Left/First Operand).
     c) Compute intermediate value: $result = A \odot B$.
     d) Push $result$ back onto $\mathcal{S}$.
3) Result: After processing all tokens, the single element remaining on $\mathcal{S}$ is the final evaluated result.

Critical Operator Asymmetry Note: Because subtraction and division are non-commutative ($A - B \neq B - A$ and $A/B \neq B/A$), the first element popped from the stack is the second operand B, while the second element popped is the first operand A. Inverting this order introduces critical calculation bugs.

3.3 Detailed Trace of Lecture Example: $[5, 2, \times, 3, -]$

Evaluating the expression from page 12 of the lecture notes:

TABLE 3
Step-by-Step Postfix Evaluation Trace for $[5, 2, \times, 3, -]$

Step	Token	Action Type	Stack State Transition ($\mathcal{S}$)	Top
1	5	Push Operand	$\emptyset \to [5]$	5
2	2	Push Operand	$[5] \to [5, 2]$	2
3	$\times$	Evaluate Operator	Pop $B = 2, A = 5 \implies 5 \times 2 = 10 \implies [10]$	10
4	3	Push Operand	$[10] \to [10, 3]$	3
5	$-$	Evaluate Operator	Pop $B = 3, A = 10 \implies 10 - 3 = 7 \implies [7]$	7

The final scalar result is 7. The algorithm completes in $O(N)$ time with $O(N)$ space.

4 The Monotonic Stack Paradigm: Nearest Smaller Element

4.1 Formal Problem Formulation

Given an array A of N integers, for every index $i \in [0 \dots N-1]$, determine the nearest smaller element to its left (Previous Smaller Element, PSE). Formally:

$$\text{res}[i] = A[j] \quad \text{such that } j < i,\ A[j] < A[i],\ \text{and } j \text{ is maximized.}$$

If no such index j exists, assign $\text{res}[i] = -1$.

4.2 The Brute Force Bottleneck

A naive implementation scans backward for each element:

```
for (int i = 0; i < N; i++) {
    res[i] = -1;
    for (int j = i - 1; j >= 0; j--) {
        if (A[j] < A[i]) {
            res[i] = A[j];
            break;
        }
    }
}
```

In worst-case descending inputs (e.g., $A = [10, 9, 8, 7, 6, 5]$), the inner loop executes $\sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}$ comparisons, yielding quadratic $O(N^2)$ time complexity. On datasets of size $N = 10^5$, $O(N^2)$ demands 10^{10} operations, causing standard CPU timeouts.

4.3 The Elimination Principle: Why Greater Elements are Useless

The breakthrough insight formulated on page 16 of the lecture notes is the Elimination Principle:

> The Elimination Lemma: If index $j < i$ and $A[j] \geq A[i]$, element $A[j]$ is mathematically dominated by $A[i]$. Element $A[j]$ can NEVER serve as the nearest smaller element for any future element $k > i$.

Physical Justification (The Skyline Shadowing Model): Consider standing at index $k > i$ and looking leftward for a smaller element:

1) $A[i]$ is physically closer to k than $A[j]$ because $i > j$.
2) $A[i]$ is strictly smaller than or equal to $A[j]$ ($A[i] \leq A[j]$).

Any future element $A[k]$ that could be satisfied by $A[j]$ would encounter $A[i]$ first, which is both shorter and nearer! Thus, $A[i]$ completely shadows $A[j]$. Once $A[i]$ enters the system, $A[j]$ becomes utterly redundant and can be permanently purged from the search state.

4.4 The Monotonic Stack Invariant

By popping all elements $\geq A[i]$, the stack maintains a strict property at all times:

> Monotonic Increasing Invariant: Elements residing in the stack are strictly ordered in increasing magnitude from bottom to top ($S[0] < S[1] < \cdots < S[\text{top}]$).

Because the stack is strictly increasing, the element located directly beneath the current element at the top of the stack is guaranteed to be its nearest smaller element!

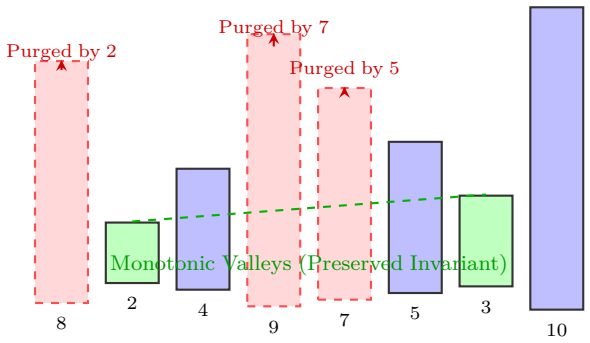


Fig. 3. The Skyline Shadowing Model: Taller preceding elements are permanently obscured by shorter subsequent elements.

4.5 Detailed Trace of Lecture Example: $A = [8, 2, 4, 9, 7, 5, 3, 10]$

We step through the exact 8-element array from pages 14–19 of the lecture notes:

$$A = [8, 2, 4, 9, 7, 5, 3, 10]$$

TABLE 4
Step-by-Step Monotonic Stack Execution Trace ($A = [8, 2, 4, 9, 7, 5, 3, 10]$)

Index i	$A[i]$	Incoming	Pop Redundant Elements (S.peek() $\geq A[i]$)	res[i]	Stack State After Push
0	8	8	Stack is empty $\implies$ no pops.	-1	[8]
1	2	2	Pop 8 ($8 \geq 2$). Stack becomes empty.	-1	[2]
2	4	4	Stack top $2 < 4 \implies$ no pops. Peek $= 2$.	2	[2, 4]
3	9	9	Stack top $4 < 9 \implies$ no pops. Peek $= 4$.	4	[2, 4, 9]
4	7	7	Pop 9 ($9 \geq 7$). Stack top becomes $4 < 7$. Peek $= 4$.	4	[2, 4, 7]
5	5	5	Pop 7 ($7 \geq 5$). Stack top becomes $4 < 5$. Peek $= 4$.	4	[2, 4, 5]
6	3	3	Pop 5 ($5 \geq 3$), Pop 4 ($4 \geq 3$). Top $= 2 < 3$. Peek $= 2$.	2	[2, 3]
7	10	10	Stack top $3 < 10 \implies$ no pops. Peek $= 3$.	3	[2, 3, 10]

Result Vector:

$$\text{res} = [-1, -1, 2, 4, 4, 4, 2, 3]$$

If storing 0-based index locations instead of values:

$$\text{res}_{\text{idx}} = [-1, -1, 1, 2, 2, 2, 1, 6]$$

Both representations resolve instantaneously in a single linear pass.

5 Amortized Complexity: Why Nested Loops Yield $O(N)$ Time

Novice software engineers frequently inspect the nested control flow:

```
for (int i = 0; i < N; i++) {
    while (!st.isEmpty() && st.peek() >= A[i]) {
        st.pop();
    }
    // ...
    st.push(A[i]);
}
```

and mistakenly classify the algorithm as quadratic $O(N^2)$ due to the inner while loop nested within the outer for loop.

5.1 The Aggregate Potential Method

To establish the true time complexity, we analyze the cumulative life cycle of each element:

1) Push Quota: Each element $A[i]$ from index 0 to $N - 1$ is pushed onto the stack exactly once. Across the entire execution of the outer loop, exactly N total push operations occur.

2) Pop Quota: An element cannot be popped unless it currently resides in the stack. Once an element is popped, it is permanently discarded and can never be re-popped. Therefore, the total number of pop operations across the entire program cannot exceed the total number of pushes:

$$\sum \text{Pops} \leq \sum \text{Pushes} = N$$

5.2 Amortized Invariant Bound

Summing all atomic operations executed by the inner while loop:

$$\text{Total Operations} = \text{Total Pushes} + \text{Total Pops} \leq N + N = 2N$$

Because $2N$ operations are shared across N iterations, the amortized time per iteration is $O(1)$.

Conclusion: The overall runtime is strictly $O(N)$ linear time, achieving the theoretical lower bound for element inspection. The auxiliary space complexity is $O(N)$ to hold the stack elements in the worst-case ascending input.

6 The Monotonic Spectrum: Four Directions & Problem Equivalence

The Nearest Smaller Element to Left is one of four symmetric variations of the Monotonic Stack pattern, summarized in Table 5:

TABLE 5
The Complete Monotonic Stack Taxonomic Spectrum

Archetype	Loop Direction	Pop Invariant	Stack Top Property	Canonical Benchmark
Previous Smaller (PSE)	Left to Right ($0 \to N-1$)	st.peek() >= A[i]	Nearest strictly smaller to left	Largest Rectangle in Histogram
Next Smaller (NSE)	Right to Left ($N-1 \to 0$)	st.peek() >= A[i]	Nearest strictly smaller to right	Final Discounted Prices
Previous Greater (PGE)	Left to Right ($0 \to N-1$)	st.peek() <= A[i]	Nearest strictly greater to left	Stock Span Problem
Next Greater (NGE)	Right to Left ($N-1 \to 0$)	st.peek() <= A[i]	Nearest strictly greater to right	Daily Temperatures

By mastering this directional spectrum, complex interview benchmarks (e.g., LeetCode 84 Largest Rectangle in Histogram, LeetCode 42 Trapping Rain Water, and LeetCode 901 Online Stock Span) reduce to standard monotonic stack traversals.

7 Production Multi-Language Reference Implementations

7.1 Java Reference Implementations

7.1.1 Balanced Delimiter Checker

```
import java.util.ArrayDeque;
import java.util.Deque;

public class BalancedParenthesesSolver {
    public static boolean isValid(String s) {
        Deque<Character> stack = new ArrayDeque<>();

        for (char c : s.toCharArray()) {
            if (c == '(' || c == '{' || c == '[') {
                stack.push(c);
            } else if (c == ')' || c == '}' || c == ']') {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if (c == ')' && top != '(') return false;
                if (c == '}' && top != '{') return false;
                if (c == ']' && top != '[') return false;
            }
        }
        return stack.isEmpty();
    }
}
```

7.1.2 Postfix Evaluator

```
import java.util.ArrayDeque;
import java.util.Deque;

public class PostfixEvaluator {
    public static int evaluate(String[] tokens) {
        Deque<Integer> stack = new ArrayDeque<>();

        for (String token : tokens) {
            if (isOperator(token)) {
                int b = stack.pop(); // Right operand
                int a = stack.pop(); // Left operand
                stack.push(applyOp(token.charAt(0), a, b));
            } else {
                stack.push(Integer.parseInt(token));
            }
        }
        return stack.pop();
    }

    private static boolean isOperator(String t) {
        return t.equals("+") || t.equals("-") || t.equals("*") || t.equals("/");
    }

    private static int applyOp(char op, int a, int b) {
        switch (op) {
            case '+': return a + b;
            case '-': return a - b;
            case '*': return a * b;
            case '/': return a / b;
            default: throw new IllegalArgumentException("Unknown operator: " + op);
        }
    }
}
```

7.1.3 Monotonic Previous Smaller Element Solver

```
import java.util.ArrayDeque;
import java.util.Deque;

public class PreviousSmallerElementSolver {
    public static int[] prevSmaller(int[] A) {
        int n = A.length;
        int[] res = new int[n];
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i < n; i++) {
            // Prune redundant elements
            while (!stack.isEmpty() && stack.peek() >= A[i]) {
                stack.pop();
            }

            res[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(A[i]);
        }
        return res;
    }
}
```

7.2 C++ Reference Implementations

7.2.1 C++ Monotonic Previous Smaller Element

```
#include <vector>
#include <stack>

using namespace std;

vector<int> prevSmaller(const vector<int>& A) {
    int n = A.size();
    vector<int> res(n);
    stack<int> st;

    for (int i = 0; i < n; i++) {
        while (!st.empty() && st.top() >= A[i]) {
            st.pop();
        }
        res[i] = st.empty() ? -1 : st.top();
```

```
        st.push(A[i]);
    }
    return res;
}
```

7.2.2 C++ Postfix Evaluation

```
#include <vector>
#include <string>
#include <stack>

using namespace std;

int evaluatePostfix(const vector<string>& tokens) {
    stack<int> st;
    for (const string& token : tokens) {
        if (token == "+" || token == "-" || token == "*" || token == "/") {
            int b = st.top(); st.pop();
            int a = st.top(); st.pop();
            if (token == "+") st.push(a + b);
            else if (token == "-") st.push(a - b);
            else if (token == "*") st.push(a * b);
            else if (token == "/") st.push(a / b);
        } else {
            st.push(stoi(token));
        }
    }
    return st.top();
}
```

7.3 Python Reference Implementations

7.3.1 Python Monotonic Previous Smaller Element

```
def prev_smaller_element(A: list[int]) -> list[int]:
    """Computes nearest smaller element to the left in amortized O(N) time."""
    stack: list[int] = []
    res: list[int] = []

    for x in A:
        while stack and stack[-1] >= x:
            stack.pop()
        res.append(stack[-1] if stack else -1)
        stack.append(x)
    return res
```

7.3.2 Python Postfix Evaluation

```
def evaluate_postfix(tokens: list[str]) -> int:
    """Evaluates Reverse Polish Notation arithmetic expression."""
    stack: list[int] = []
    for token in tokens:
        if token in {"+", "-", "*", "/"}:
            b = stack.pop()
            a = stack.pop()
            if token == "+": stack.append(a + b)
            elif token == "-": stack.append(a - b)
            elif token == "*": stack.append(a * b)
            elif token == "/": stack.append(int(a / b))
        else:
            stack.append(int(token))
    return stack.pop()
```

8 Systems Engineering & Industrial Architectures

The stack data structure is an indispensable pillar of modern software engineering:

1) JVM Operand Stack and Execution Frames: In low-level bytecode execution, Java Virtual Machine instructions operate entirely on an evaluation operand stack rather than hardware CPU registers, achieving cross-platform architectural independence.
2) Browser History Dual-Stack Engine: Navigation engines manage web session travel using dual stacks: a Back Stack and a Forward Stack. Pushing URLs onto the back stack allows instantaneous $O(1)$ retreat, while navigating forward pops from the forward stack.
3) Software Undo/Redo Frameworks: Modern IDEs (e.g., VS Code) and graphic suites execute the Gang-of-Four Command Pattern via inverse command execution logged onto an undo stack.
4) Compiler Syntax Trees and Precedence Parsing: Production compilers utilize Shift-Reduce parsers and monotonic operator precedence stacks to construct Abstract Syntax Trees (ASTs) during lexical and semantic analysis.

9 Conclusion

In this research paper, we formalized the foundational theory, algorithmic mechanics, and production applications of the Stack data structure. We established the LIFO behavioral contract over contiguous arrays and dynamic buffers, rigorously addressing boundary invariants such as stack underflow and overflow. We proved how the Last Opened Bracket (LOB) invariant governs the $O(N)$ syntax validation of balanced multi-character delimiters.

Furthermore, we explored parenthesis-free deterministic computation through Postfix (RPN) arithmetic evaluation, emphasizing operand popping asymmetry. Finally, we demystified the Monotonic Stack paradigm through the Elimination Lemma, proving via the Skyline Shadowing Model why greater elements can be safely pruned. Aggregate amortized potential analysis verified that nested while loops achieve optimal $O(N)$ linear time complexity. Complete implementations across Java, C++, and Python alongside industrial runtime architectures solidify the stack as a masterclass in algorithmic efficiency.